Object-Oriented Programming Final Project Guidelines

Course Objective

The Final Project is designed to evaluate your understanding and application of Object-Oriented Programming (OOP) principles through the development of a software application or game. The primary focus of this project is the quality of your object-oriented design, code organization, and implementation.

Project Options

Each group must choose one (1) of the following project categories.

Option 1. Business or Management System

Develop a desktop application that solves a real-world business or organizational problem.

Suggested Topics

Choose one of the following:

- Library Management System
- Restaurant Ordering System
- Hotel Reservation System

Requirements

- Java with Swing GUI
- No database (all data must be stored in memory using appropriate data structures)
- Demonstrate proper OOP implementation
- Functional and user-friendly interface

Option 2. Game Development

Design and develop your own original 2D desktop game using Java and Swing.
The game concept is entirely up to your group, provided it is appropriate for the course and approved by the instructor.

Examples include, but are not limited to:

- Arcade Games
- Puzzle Games
- Platform Games

Students are encouraged to create their own gameplay mechanics instead of recreating existing games exactly.

Requirements

- Java with Swing GUI
- No game engines (e.g., Unity, Unreal, Godot)
- No database
- The game must demonstrate object-oriented design through game objects, behaviors, and interactions.

Programming Requirements

All projects must be developed using Java with Swing for the graphical user interface.
The following are not allowed:

- Database systems (MySQL, SQLite, PostgreSQL, MongoDB, etc.)
- JavaFX
- Web-based applications
- Mobile applications
- Game engines

All information should be managed using appropriate Java collections and object-oriented classes.

Required OOP Concepts

Your project must clearly demonstrate the following concepts:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction
- Composition and/or Aggregation
- Appropriate use of interfaces or abstract classes
- Exception Handling

Simply including these concepts is not enough. They must be applied meaningfully within your project's design.

Groupings: You are allowed to choose your group members, but you must strictly follow the required number of members per group. Coordinate with other groups to ensure that the total number of groups is accurate and no student is left out.

Section A: 32 Students

- 4 groups of 5 members
- 2 groups of 6 members
- **Total: 6 groups**

Section B: 38 Students

- 6 groups of 5 members
- 2 groups of 4 members
- **Total: 8 groups**

Submissions

Each group must submit the following:

1. **GitHub Repository Link**
    - Use GitHub as your **main collaboration platform** for your project.
    - Ensure that all commits clearly show each member's contributions.
    -
2. **Google Drive Link**
    - Upload a **ZIP file** of your entire project folder.
    - **File Name Format:** LastNameOfSubmitter_FirstName.zip
3. **Java GUI Final Project Document**
    - Submit your final project documentation in the required format.
4. **YouTube Video Link**
    - Upload your **video demonstration** of the project to YouTube.
    - Include the link in your submission.

Note: Only one representative from your group should submit all the required files in the designated Google Classroom post for this project.

Video Requirements

Your video should include the following:

1. **Application Walkthrough / Demonstration**
    - Show how your Java GUI project works from a user's perspective.
    - Demonstrate the main features and functionality.
2. **Code Explanation**
    - Each member must clearly explain the specific parts of the **code** they worked on.
    - Highlight how **OOP concepts** (Encapsulation, Inheritance, Polymorphism, Abstraction) were applied.
    - Ensure that each member's contribution is shown and explained in the video.

Note: The grade received by your group will also serve as your individual grade. Additionally, do not include the names of members who did not contribute to the project.

Project Rubric:

Category	Weight	Excellent (90-100%)	Good (80-89%)	Satisfactory (70-79%)	Needs Improvement (0-69%)
OOP Implementation	25%	Demonstrates a strong understanding of OOP. - Uses encapsulation effectively; data is private, and methods control access. - Correctly implements inheritance/abstraction (e.g., Product, Room, GameObject). - Uses polymorphism where appropriate (e.g., a list of GameObjects). - Classes have clear, single responsibilities.	Correctly applies OOP concepts. - Most data is private, but encapsulation may be incomplete. - Required inheritance/abstraction is present and functional. - Classes are mostly well-defined, with minor mixing of concerns.	Attempts to apply OOP, but with errors. - May use many public fields or static variables improperly. - Inheritance/abstraction may be missing or incorrectly used. - Logic is often mixed with data classes (e.g., all logic inside the Book class).	Fails to apply OOP concepts. - Logic is mostly in one or two "God classes." - No clear use of encapsulation, inheritance, or polymorphism. - Classes are just simple data containers.
Functionality & Feature Completion	20%	All core features are fully functional and bug-free. - The application runs smoothly and is stable. - All specified logic (e.g., calculateTotal(), checkOutBook(), collision detection) works correctly. - Handles basic user errors gracefully.	All core features are implemented, but with minor bugs. - The application is usable, but minor issues exist (e.g., a calculation is slightly off, an item isn't removed correctly). - May crash on unexpected user input.	Most core features are implemented, but some are missing or non-functional. - The application runs but has noticeable bugs that impede core functionality. - Key logic is incomplete or incorrect.	The application is non-functional. - Fails to launch, crashes immediately, or the majority of core features are missing or broken.
GUI Implementation (Swing)	15%	GUI is well-designed, functional, and responsive. - Uses appropriate components and layout managers (e.g., JTabbedPane, CardLayout, GridLayout) effectively. - Event handling is clean and works as expected. - The GUI dynamically updates as needed	GUI is functional and meets all requirements. - Uses appropriate components, but layout might be simple or rigid (e.g., over-reliance on FlowLayout or null layout). - Event handling works, but the code may be disorganized.	GUI is functional but flawed. - The layout is confusing, or inappropriate components are used. - GUI does not update dynamically (e.g., requires a manual refresh). - Event handling is buggy.	GUI is non-existent, non-functional, or does not use Swing. - Components are not visible, or the app is console-only.

		(e.g., cart total, room color).			
Code Quality & Conventions	10%	Code is clean, readable, and follows all Java conventions. - Consistent and meaningful naming (e.g., Book, memberID, checkOutBook()). - Code is well-formatted and easy to read. - Good use of comments to explain why, not what.	Code is readable and follows most conventions. - Naming is mostly consistent. - Formatting is generally good. - Some comments are present, but they may be sparse.	Code is difficult to read and inconsistent. - Mixed naming conventions (e.g., book, Member_ID). - Poor formatting (e.g., bad indentation). - Few or no comments.	Code is unreadable. - Ignores all Java conventions. - No formatting, no comments. - Logic is impossible to follow.
External Documentation	15%	Provides clear, complete, and professional documentation. - A well-written User Guide explains all features.	Documentation is present and covers most features. - User Guide is understandable.	Documentation is incomplete. - Instructions are hard to follow.	Documentation is missing, auto-generated without content, or completely unhelpful.
Video Demonstration	15%	Video is clear, concise, and professional. - Demonstrates all core features working correctly. - Audio and video quality are high. - Each group member speaks and clearly explains a part of the code they worked on, showing a deep understanding. Within 15 mins.	Video is clear and demonstrates most features. - Demonstrates most core features, but may skip some or show minor bugs. - Group members speak, but the code explanation may be superficial or unclear. 15-20 mins	Video is lacking in quality or content. - Hard to hear or see. - Demonstrates only a few features. - Only one person speaks, or there is no explanation of the code. 20-25 mins	Video is missing, unwatchable, less than 1 minute, or does not show the project running. Or the video exceeds More than 25 mins

Total: 100%

Documentation Guidelines:

Project Topic:

GROUP MEMBERS (Last Name, First Name)	**CONTRIBUTION** Indicate which parts of the project (video demonstration and source code) you personally contributed to, and briefly describe your specific role or tasks.
1.	
2.	
3.	
4.	
5.	

Remove this when submitting (Make a copy of this document)

Format:

12 px

Times New Roman

1.5 spacing

Black color

USER DOCUMENTATION (EXTERNAL)

- Write a simple guide for the people who will use your application or play your game.
- Explain what users need to do, which buttons or controls to use, and what happens afterward.
- **No installation guide or explanation of the source code is needed.**

USER DOCUMENTATION TEMPLATE

Project Name: [Your Project Name]
Project Type: [Business or Management System / Game]

1. **INTRODUCTION**

Write a short description that answers these questions:

- What is your application or game?
- Who is it for?
- What are its main features?

For a system, explain what tasks it helps users complete.

For a game, explain what the player needs to accomplish.

2. **MAIN SCREEN**

Describe the important parts of the screen and how to use them.

For a system:

- What menus, tabs, forms, or buttons are available?
- How does the user move between sections?

For a game:

- What options are available on the main menu?
- How does the player start the game?
- Where can the player see the score, lives, or timer, if included?

Optional: Add a screenshot with labels.

3. **HOW TO USE THE SYSTEM OR PLAY THE GAME**

Choose the option that matches your project. Remove the other option.

OPTION A: BUSINESS OR MANAGEMENT SYSTEM

Write instructions for each main feature using numbered steps.

Example:

3.1 Adding a New Book

1. Open the Book Management tab.
2. Enter the title, author, and ISBN.
3. Click Add Book.
4. The new book appears in the list.

Repeat this format for your other features, such as:

- Searching for a record.
- Editing or deleting a record.
- Placing an order.
- Making a reservation.

Use the actual features and button names from your application.

OPTION B: GAME

3.1 Objective and Rules

Explain:

- What the player must do.
- How to win or complete a level.
- What causes the player to lose.
- Any important rules, obstacles, or special items.

3.2 Controls

List each control and its action.

Example:

- Left Arrow: Move left.
- Right Arrow: Move right.
- Spacebar: Jump.

Replace these examples with your game's actual controls.

3.3 How to Play

Write simple steps.

1. Click [button name] to start.
2. Use [controls] to [action].
3. Complete [objective].
4. Avoid [obstacles or hazards].
5. The game ends when [winning or losing condition].

Explain scores, lives, levels, or time limits if your game includes them.

3.4 Other Actions

Explain how to do the following, if available:

- Pause and resume.
- Restart or try again.
- Return to the main menu.
- Exit the game.

4. IMPORTANT REMINDERS

Tell users what they need to know, such as:

- Whether data or game progress is saved after closing.

- What happens when they delete a record or restart a game.
- Any limits that affect how they use the application or game.

BEFORE SUBMITTING

- Replace all placeholders with your project details.
- Remove sections that do not apply.
- Use the exact button names and controls from your project.
- Check that the instructions match how your application or game works.